

to run pre-existing TensorFlow models right in the browser.

- *Retrains existing models:* Retrains pre-existing ML models using sensor data connected to the browser or other client-side data.

## TensorFlow.js architecture

TensorFlow.js, as the name suggests, is based on TensorFlow, with a few exceptions specific to the JS environment. This library comes with the following two sets of APIs:

- The Ops API facilitates lower-level linear algebra operations such as matrix multiplication, tensor addition and so on.
- The layers API, similar to the Keras API, provides developers high-level model building blocks and best practices with the emphasis on neural networks.

Figure 1 depicts the TensorFlow.js architecture.

## TensorFlow.js APIs

TensorFlow.js is powered by WebGL and provides a high-level layers API for defining models, and a low-level API for linear algebra and automatic differentiation. It supports TensorFlow SavedModels and Keras models. Listed below are the TensorFlow.js APIs.

**TensorFlow.js Core API:** This provides low-level, hardware-accelerated linear algebra and the API for automatic differentiation. It's a flexible low-level API for neural networks and numerical computation. TensorFlow.js can be installed via Yarn or npm.

**TensorFlow.js layers:** This is a high-level API built on TensorFlow.js Core, enabling users to build, train and execute deep learning models in the browser. It is modelled on Keras and `tf.keras`, and so can load models saved from those libraries.

There are three ways to import TensorFlow.js layers. First, users can access these layers via the Union package between TensorFlow.js Core

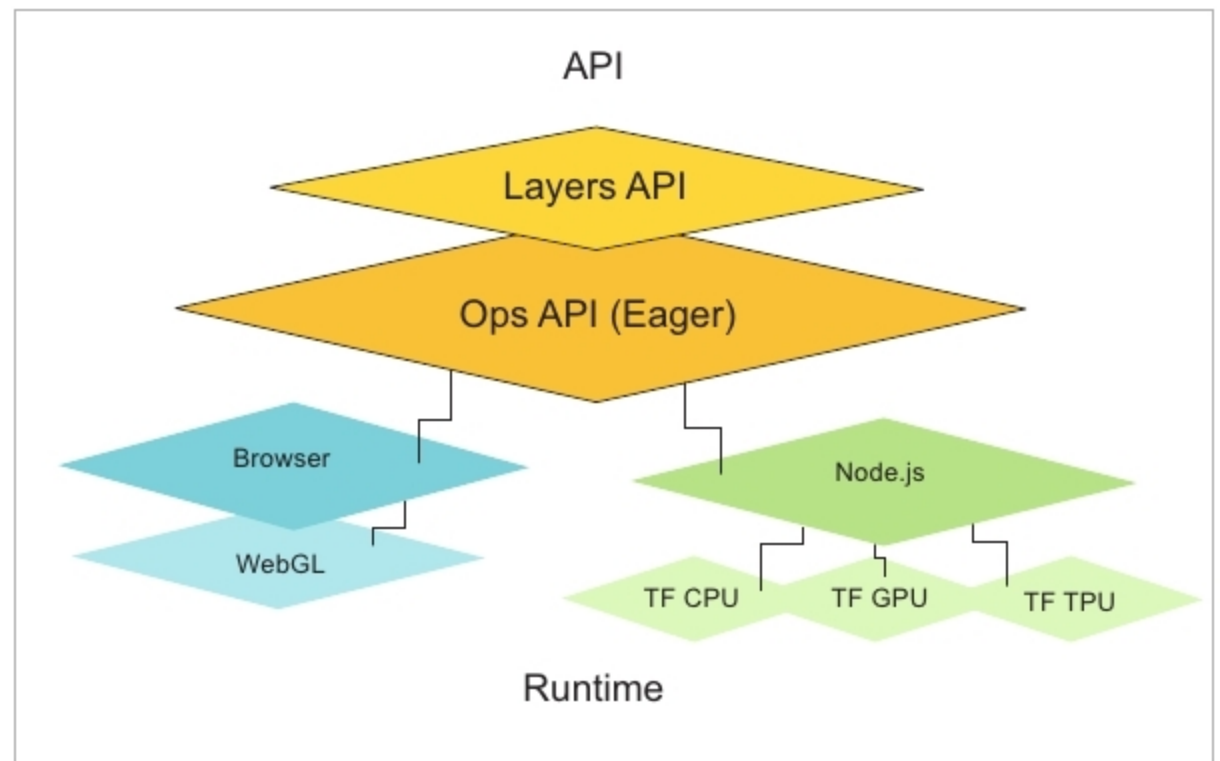


Figure 1: TensorFlow.js architecture

and the layers. Second, they can be imported as a module: `@tensorflow/tjs-layers`. And third, they can be imported as standalone via `unpkg`.

**TensorFlow.js data:** This provides simple APIs to load and parse data from the disk or over the Web in a variety of formats, and to prepare that data for use in ML models like filter, map, shuffle and batch.

There are two ways to import TensorFlow.js data:

- Access TensorFlow.js data via the Union package: `@tensorflow/tjs`
- Get TensorFlow.js data as a module: `@tensorflow/tjs-data`.

**TensorFlow.js Converter:** This is an open source library to load a pre-trained TensorFlow SavedModel or TensorFlow Hub module into the browser and run the interface via TensorFlow.js.

In order to import the model, a two-step process needs to be followed:

- The Python `pip` package has to convert a TensorFlow SavedModel or TensorFlow Hub module to a Web friendly format; and
- The JavaScript API is used for loading and running inference.

**TensorFlow.js Vis:** This is a small library for in-browser visualisation intended for use with TensorFlow.js.

It has a set of visualisations for model behaviour, a set of high-level functions for visualising objects specific to TensorFlow.js and a way to organise visualisations of model behaviour that won't interfere with the Web application. The library also aims to be flexible and make it easy for you to incorporate custom visualisations using tools of your choice, such as `d3`, `Chart.js` or `Plotly.js`.

**TensorFlow.js AutoML:** This consists of a set of APIs to load and run models produced by AutoML Edge, the models of which are of two types: image classification and object detection.

The AutoML image classification model will output the following sets of files:

- `model.json`, the model topology
- `dict.txt`, a newline-separated list of labels
- One or more of `*.bin` files which hold the weights

The AutoML Object detection model will output the following sets of files:

- `model.json`, the model topology
- `dict.txt`, a newline-separated list of labels
- One or more of `*.bin` files which hold the weights